

Лабораторная работа №3

Деревья решений в задачах классификации и регрессии. ROC-кривая

Логические алгоритмы принятия решений

Логические алгоритмы принятия решений дают ответ на поставленную задачу исходя из результата применения набора утверждений к объекту, поданному на вход такой модели.

Приведем пример, нам нужно решить задачу бинарной классификации: придет ли человек на День Радиофизика (C_0 — не придет, C_1 — придет)? Ответ мы будем строить на применении к конкретному человеку следующих предикатов:

$$f_1(x) = \text{“}x \text{ — студент РФУКТ”}$$

$$f_2(x) = \text{“}x \text{ любит халаву”}$$

$$f_3(x) = \text{“}x \text{ — выпускник РФУКТ”}$$

$$f_4(x) = \text{“}x \text{ любит свой } Alma Mater\text{”}$$

Тогда окончательное решение мы вынесем на основе конъюнкций вышеописанных предикатов. Для отнесения человека к классу C_1 , то есть он придет на мероприятие, одно из следующих комплексных утверждений о нем должно быть истинным

$$R(x) = f_1(x) \wedge f_2(x) = \text{“}x \text{ — студент РФУКТ и } x \text{ любит халаву”}$$

$$R(x) = \overline{f_1(x)} \wedge f_3(x) \wedge f_4(x) = \text{“}x \text{ — не студент РФУКТ и } x \text{ — выпускник РФУКТ и любит свой } Alma Mater\text{”}$$

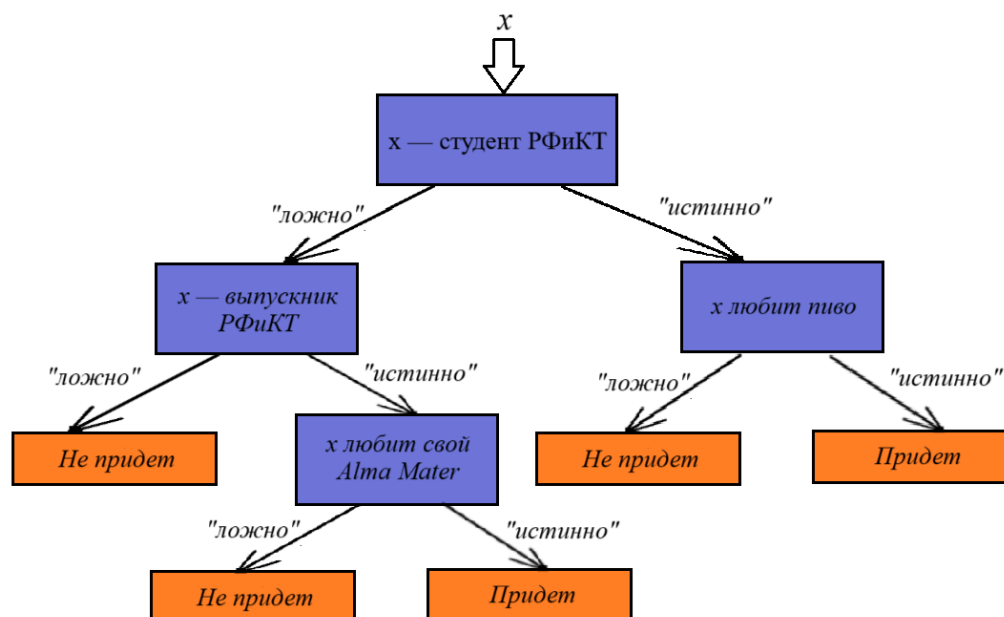
Чтобы определить, что человек не явится на празднество (класс C_0) требуется истинность одного из нижеперечисленных утверждений

$$R(x) = f_1(x) \wedge \overline{f_2(x)} = \text{“}x \text{ — студент РФУКТ и } x \text{ не любит халаву”}$$

$$R(x) = \overline{f_1(x)} \wedge \overline{f_3(x)} = \text{“}x \text{ — не студент РФУКТ и } x \text{ — не выпускник РФУКТ”}$$

$$R(x) = \overline{f_1(x)} \wedge f_3(x) \wedge \overline{f_4(x)} = "x \text{ — не студент РФиКТ и } x \text{ — выпускник РФиКТ и } x \text{ не любит свой Alma Mater}"$$

Можно заметить, что в конъюнкциях утверждения об объекте x выстроены в некоем порядке: каждый последующий предикат зависит от результата предыдущего. Тогда принятие решения представим в следующем виде:



Таким образом мы приходим к одному из *логических алгоритмов принятия решений* — **бинарному дереву решений**.

Деревья решений

Дерево решений — модель принятия решений, использующееся в машинном обучении, кибербезопасности, анализе данных и статистике, в частности решает задачи классификации и регрессии.

Для моделей машинного обучения в **бинарном дереве решений** каждый узел дерева представляет собой предикат о некотором признаке объекта, например, “*возраст больше 30*”. От узла отходят две ветви, характеризующие результат утверждения — “*истинно*”/“*ложно*”. А листья дерева содержат итоговые решения: для классификации — метку класса, для регрессии — числовое значение. Узел, не имеющий предков, называется **корневым**.

В общем же случае дерево решений не обязательно должно быть бинарным и внутренний узел дерева может иметь более чем две ветви. Однако на практике такие модели очень редко находят применение.

Если наблюдение x имеет n признаков и его вектор записывается в виде $x = [x_1, x_2, \dots, x_n]$, то предикат для j -го вещественного (непрерывного) признака в k -ом узле решающего дерева можно представить в виде выражения

$$B_k(x) = [x_j \leq a_k]$$

где $[]$ — нотация Айверсона, а a_k — некоторый порог, с которым сравнивается j -ый вещественный признак.

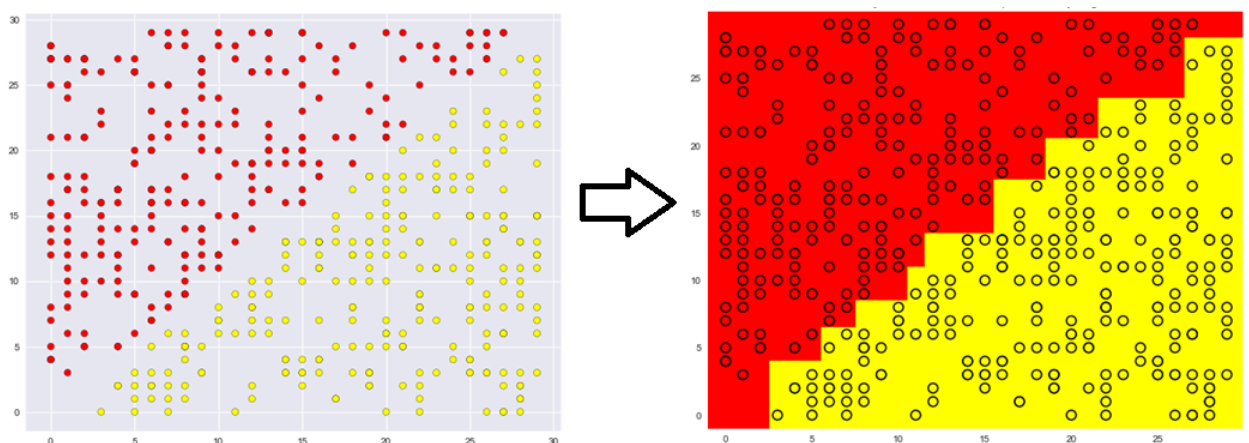
Для дискретных (категориальных) признаков предикат k -го узла проверяет принадлежность j -го признака к подмножеству его значений, то есть

$$B_k(x) = [x_j \in X_j^k]$$

где X_j^k — подмножество множества всех значений j -го признака для k -го узла.

Таким образом, алгоритм выглядит следующим образом: начиная с корневого узла ($k = 0$), для наблюдения x в k -ом вершине вычисляется выражение $B_k(x)$, соответствующее предикату этой вершины. На основании полученного результата выбирается следующий узел дерева: $B_k(x) = 1$ — узел соответствующий ветви “истинно”, $B_k(x) = 0$ — ветви “ложно”. Алгоритм продолжается пока не дойдет до листовой вершины с ответом на поставленную задачу.

Графически решение бинарной классификации с двумя признаками можно представить следующим образом:



Оси соответствуют признакам x_1 и x_2 . Дерево же решений с помощью набора предикатов для этих признаков построила границу между двумя областями разных классов.

Нераскрытыми остаются вопросы: как строить такие деревья? Какие признаки и пороги выбирать для предикатов в узлах? Как создать наиболее оптимальное дерево?

Здесь, при решении задач классификации и регрессии, также используются *функции стоимости*. Однако их вид и применение отличны от тех, что используются в линейных моделях. Начнем с разбора построения (де факто обучения) дерева решений для задачи классификации, которая в целом может быть многоклассовой.

Решение задачи классификации деревом решений

Одной из характеристик графов в виде деревьев является их *глубина*. Так как решающие деревья строятся на основе обучающей выборки, то чем больше глубина такого дерева, тем больше количество предикатов, описывающих обучающую выборку, а следовательно тем сильнее модель подстраивается под обучающие данные, т.е. переобучается. Примером, когда решающее дерево максимально переобучено, может служить дерево, в котором каждому листу соответствует только одно наблюдение из обучающей выборки.

Значит, требуется найти дерево хорошо решающее поставленную задачу классификации и имеющее небольшую глубину в целях лучшей *обобщающей способности*.

В случае *бинарного решающего дерева* в каждом узле обучающая выборка делится на две подвыборки на основе предиката. Для достижения меньшей глубины решающего дерева, естественной эвристикой является такое разбиение выборки, попавшей в конкретный узел, после которого в одной из подвыборок большинство наблюдений будет относиться к одному классу. Это позволит сделать менее глубокое поддерево уже для разбиения такой подвыборки, так как объекты в ней более однообразны с точки зрения классов.

Тогда в качестве меры разнообразия выборки можно использовать *энтропию*. Она характеризует хаотичность системы, то есть в случае построения решающего дерева — чем больше объектов разных классов в

выборке, тем больше энтропия; чем меньше разнообразие — тем более система определена и тем меньше энтропия. Нулевая энтропия соответствует тому факту, что в выборке присутствуют экземпляры только одного класса. Формула информационной энтропии выглядит следующим образом

$$S = - \sum_{i=0}^C p_i \log_2 p_i$$

где C — общее количество целевых классов, а p_i — вероятность появления объекта i -го класса в выборке.

В рамках решаемой задачи нам требуется, чтобы после разбиения выборки, одна из подвыборок должна будет иметь меньшую энтропию, нежели была до этого. Из теории информации мы знаем, что *информация* — это разница энтропий системы в начальном и конечном состояниях. Поэтому в качестве *функции стоимости для предиката* вводят **информационный прирост (информационный выигрыш, information gain)** равный разнице энтропий выборки, разбиваемой в k -ом узле, и взвешенных энтропий её подвыборок

$$IG_k(j, a_k) = S_0 - \left(\frac{N_1}{N} S_1 + \frac{N_2}{N} S_2 \right), \text{ если } j\text{-й признак} \text{ — вещественный}$$

$$IG_k(j, X_j^k) = S_0 - \left(\frac{N_1}{N} S_1 + \frac{N_2}{N} S_2 \right), \text{ если } j\text{-й признак} \text{ — категориальный}$$

где S_0 и N — энтропия выборки попавшей в k -ую вершину и количество объектов в этой выборке соответственно, S_1 и S_2 — энтропии подвыборок, а N_1 и N_2 — количество объектов в подвыборках.

Если вершина дерева разбивает выборку на более чем две подвыборки, то *информационный выигрыш* принимает общий вид

$$IG_k(j, a_k) = S_0 - \sum_{i=1}^Q \frac{N_i}{N} S_i$$

$$IG_k(j, X_j^k) = S_0 - \sum_{i=1}^Q \frac{N_i}{N} S_i$$

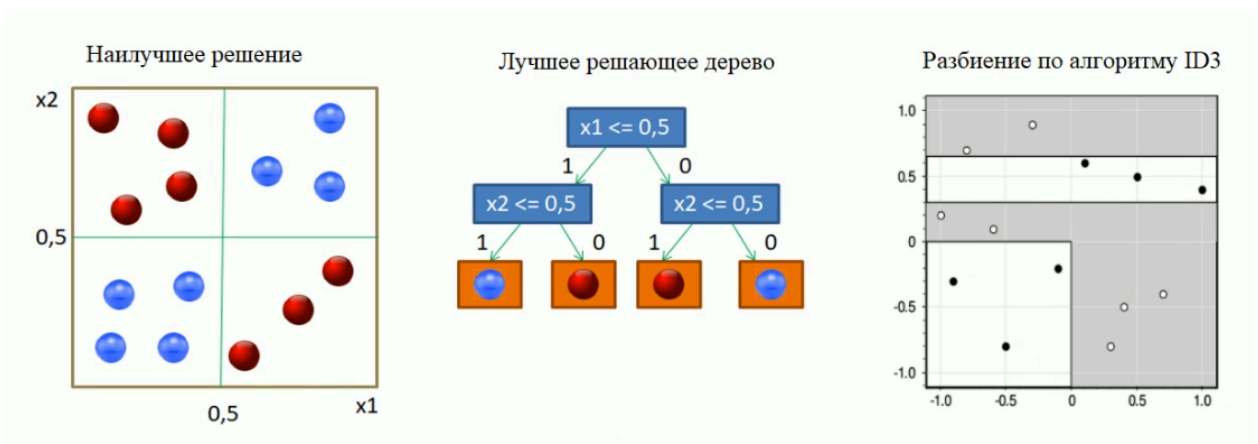
где Q — число исходящих из k -ой вершины ветвей.

Из формул видно, что *информационный прирост* зависит от выбираемого для утверждения в k -ом узле j -го признака и от подбираемого к нему порога a_k или подмножества значений X_j^k , в зависимости от вида этого признака. Однако, в отличие от функций стоимости в линейных моделях, информационный выигрыш *максимизируется*.

Таким образом, обучение дерева решений для задачи классификации сводится к построению вершин дерева путем подбора порога или подмножества для предиката в вершине на основе *информационного выигрыша*, полученного после разбиения этим предикатом части обучающей выборки, попавшей в данную вершину.

На этом подходе основан алгоритм построения решающего дерева для задачи классификации **ID3 (Iterative Dichotomiser 3)**. Его относят к числу жадных алгоритмов, так как он находит наиболее выгодное решение на каждом шаге работы. То есть, в случае построения дерева, *определяет предикат в k -ом узле из условия получения максимального информационного выигрыша IG_k* после разбиения на подвыборки.

Такой подход к построению вершин не всегда способствует самому оптимальному решению и может приводить не к самой лучшей обобщающей способности модели. Такую ситуацию можно представить в виде следующего примера



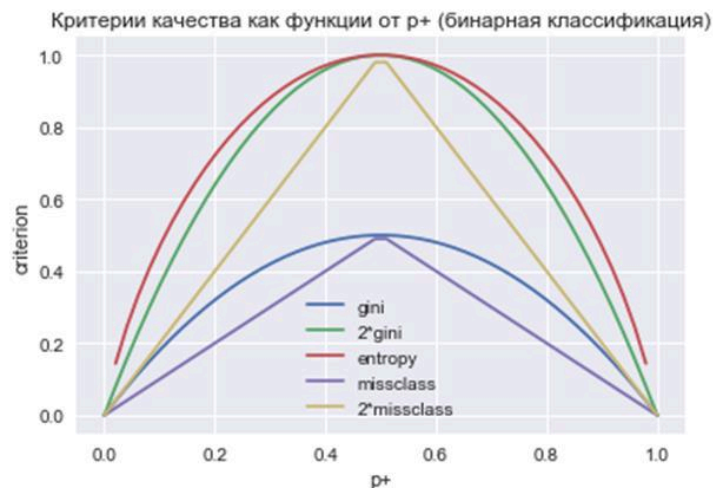
Однако алгоритм ID3 относительно прост в реализации и поэтому является базой для других алгоритмов, являющихся модификациями первого.

Одной из модификаций является алгоритм **CART (Classification and Regression Tree)** — предназначен для построения бинарного дерева решений и, в отличие от ID3, при подсчете *информационного выигрыша* использует вместо энтропии *критерий Джини (Gini Impurity)*

$$S = 1 - \sum_{i=0}^C p_i^2$$

где C — общее количество целевых классов, а p_i — вероятность появления объекта i -го класса в выборке.

Критерий Джини вычислительно менее затратен, а его удвоенный график практически повторяет энтропию:



Именно этот алгоритм по умолчанию используется “под капотом” моделей деревьев решений из библиотеки *scikit-learn*, о чем можно узнать на [этой странице](#). Работа с [*DecisionTreeClassifier*](#) выглядит также, как и с другими моделями библиотеки *scikit-learn*

```
from sklearn.tree import DecisionTreeClassifier
dt_classifier_model = DecisionTreeClassifier()
```

Все еще не был затронут вопрос определения вершины как листовой, то есть той, в которой содержится ответ на поставленную задачу. Самым очевидным критерием для терминального узла дерева можно взять нулевую энтропию у выборки, попавшей в данный узел. Но такой критерий останова может привести к достаточно глубокому дереву, если в обучающей выборке будут наблюдаться выбросы, которые алгоритм будет пытаться отделить для достижения нулевой энтропии. Поэтому, в целях избежания переобучения, на структуру дерева вводят ограничения и применяют менее строгие критерии останова для листовых узлов:

- Энтропия в узле меньше некоторого заданного значения
- Число попавших в вершину объектов меньше заданной величины

- Вероятность правильной классификации объекта больше заданного значения
- Достигнуто заданное максимальное количество листьев в дереве
- Достигнута заданная максимальная глубина дерева

Список ограничений на этом не заканчивается, однако это основные используемые на практике. Также в процессе построения дерева обычно используют несколько критерий останова одновременно: выбор происходит из расчета получения хорошей обобщающей способности итогового дерева.

Таким образом, критерии формирования листовых вершин можно рассматривать как *эвристики регуляризации* решающего дерева. Можно выделить два типа таких эвристик в зависимости от того, когда их применяют к дереву:

- *pre-pruning (early stopping)* — применение критериев останова в процессе построения дерева (то, что было перечислено ранее)
- *pruning (усечение, стрижка)* — изменение структуры дерева уже после его построения

Pre-pruning — это, де факто, установка гиперпараметров модели до её обучения. Для модели *DecisionTreeClassifier* это делается во время её создания. Пример установки максимальной глубины дерева и максимального числа листьев

```
dt_classifier_model = DecisionTreeClassifier(
    max_depth=4, max_leaf_nodes=10)
```

Больше про применимые к *DecisionTreeClassifier* гиперпараметры можно узнать [здесь](#).

Pruning

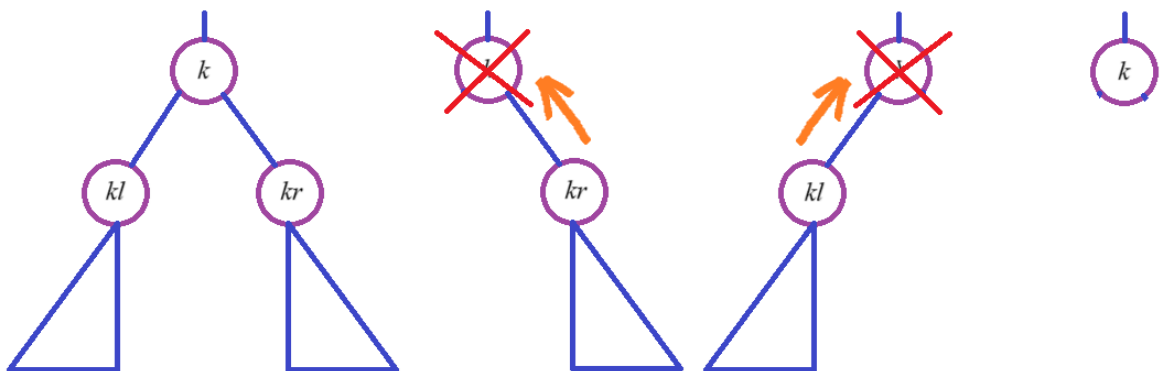
Стрижка дерева, после его обучения, происходит уже с использованием *тестовой* выборки, так как главной задачей усечения дерева является — *повышение обобщающей способности модели*.

Подготовительным этапом выступает подсчет количества объектов *тестовой* выборки, попавших в каждую из внутренних вершин дерева. Обозначим множество тестовых наблюдений добравшихся до k -ой вершины как X_k^{test} , где $X_k^{test} \subset X^{test}$.

Первый метод усечения дерева — “обрезка” внутренних узлов, в которые не попал ни один объект из тестовой выборки, т.е. $X_k^{test} = \emptyset$. Их удаление возможно, так как высока вероятность, что поддерево на основе такого узла описывает детали обучающей выборки, например, выбросы наблюдаемые в данных. Тогда такой узел можно заменить на лист с меткой класса, соответствующей наибольшему числу объектов одного класса из *обучающей* выборки, которые попали в эту вершину.

Если же X_k^{test} оказалось не пустым, то для дерева на основе k -ой внутренней вершины рассчитывается четыре случая ошибки классификации *Error Rate* для X_k^{test} :

1. Дерево на основе k -ой вершины без изменений
2. Левое поддерево k -ой вершины обрезается и вершина заменяется на правое поддерево
3. Правое поддерево k -ой вершины обрезается и вершина заменяется на левое поддерево
4. Вершина превращается в листовую таким же способом, как и в прошлом методе усечения



На основе рассчитанных *Error Rate*, k -ый внутренний узел принимает вид, для которого ошибка классификации оказалась наименьшей.

Такие способы усечения эффективно применяются при использовании жадных алгоритмов построения деревьев, так как при них высока вероятность переобучения модели.

Работа с пропущенными значениями

Одной из особенностей классифицирующих решающих деревьев является возможность работать с наблюдениями, у которых есть пропущенные значения для некоторых признаков.

Представим, что наблюдение x было подано на вход обученного дерева решений и дошло до внутренней вершины, которая работает с j -ым признаком, но значение x_j у наблюдения оказалось пустым. В таком случае следующий шаг выбирается на основе наиболее вероятного исхода для обучающей выборки.

Это значит, что для каждой внутренней вершины рассчитывается вероятность попадания в его дочернюю вершину на основе *обучающей* выборки. Например, в k -ую внутреннюю вершину бинарного решающего дерева попало подмножество объектов обучающей выборки U , а в ее левую и правую дочерние вершины — U_l и U_r соответственно. Тогда вероятность попадания в дочернюю вершину рассчитывается как

$$p_l = \frac{|U_l|}{|U|}$$

$$p_r = \frac{|U_r|}{|U|}$$

Примерно таким же способом можно рассчитать условную вероятность отнесения наблюдения x к конкретному классу y при попадании в k -ую вершину, а именно

$$P(y | k) = \frac{1}{|U|} \sum_{x_i \in U} [y_i = y]$$

где x_i — наблюдение из обучающей выборки, а y_i — метка класса, выданная **обученной моделью** для наблюдения x_i .

Вероятностное представление классификации и задача ранжирования

Вероятностное представление задачи классификации было частично затронуто в прошлой лабораторной работе, где описывалась *логистическая регрессия*. Она выдавала метку класса по факту преодоления выходным значением некоторого порога. Так как значения сигмоиды заперты между 0 и 1, то мы предположили, что выходное значение гипотезы — это вероятность отнесения к целевому классу.

Теперь же выведем логистическую регрессию через немного другой подход. Пусть изначальной задачей стоит получение такой модели, которая

бы возвращала вероятность отнесения к целевому классу при бинарной классификации. Тогда гипотеза будет выглядеть следующим образом

$$a(w, x) = P(y = 1 \mid x, w)$$

Задачей стоит подбор таких параметров w , чтобы модель выдавала наиболее **правдоподобные вероятности** отнесения к целевому классу поданных на вход объектов. Вероятность отнесения к противоположному классу — $1 - P(y = 1 \mid x, w)$. Из этого следует, что модель должна *максимизировать функцию правдоподобия* для обучающей выборки:

$$\prod_{i=0}^l P(y^i \mid x^i, w) \rightarrow \max$$

После нехитрых преобразований, описанных в предыдущей лабораторной работе, приходим к *функции потерь* следующего вида:

$$-\sum_{i=0}^l \log(P(y^i \mid x^i, w)) \rightarrow \min$$

Можно заметить, что *logloss* очень похожа на функцию стоимости как по виду, так и по поставленной задаче минимизации:

$$Q(w) = \sum_{i=0}^l L_i(w) \rightarrow \min$$

В качестве функции потерь для задачи классификации можно взять *логарифмическую функцию*, зависящую от *отступа*:

$$L_i(M) = \log_2(1 + e^{-M})$$

$$M_i = \langle w, x^i \rangle \cdot y^i$$

Тогда, после ряда преобразований, получим формулу для получения условной вероятности отнесения объекта к классу y

$$-\sum_{i=0}^l \log(P(y^i \mid x^i, w)) = \sum_{i=0}^l \log(1 + e^{-M})$$

$$\log(P(y^i \mid x^i, w)) = -(y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)) = -\log(1 + e^{-M})$$

$$P(y^i \mid x^i, w) = (1 + e^{-M})^{-1}$$

В результате мы опять пришли к *логистической регрессии*. Но её немного измененный вид позволяет увидеть, что выходная вероятность отнесения к классу y зависит от отступа M . Отступ же характеризует то, насколько далеко в области класса y объект x отстоит от разделяющей гиперплоскости.

В деревьях же решений вероятностный выход можно получить рассчитав описанную ранее условную вероятность отнесения наблюдения x к конкретному классу y при попадании в вершину

$$P(y \mid k) = \frac{1}{|U|} \sum_{x_i \in U} [y_i = y]$$

где k — номер листовой вершины, в которую попало наблюдение x ; а U — множество объектов обучающей выборки, также попавших в этот лист.

В случае вычисления вероятности *деревом решений*, *цепочка истинных предикатов*, приведших объект x в лист, имеет такое же смысловое значение как и *отступ*, при вычислении вероятности в *логистической регрессии*.

Для получения вероятностных предсказаний любой моделью МО из *scikit-learn* используйте метод *predict_proba()*

```
y_proba = ml_model.predict_proba(X_test)
```

Метод вернет для каждого наблюдения из X_test список вероятностей его отнесения к каждому классу в порядке их меток. Порядок можно классов можно узнать через атрибут *classes_*

```
print(ml_model.classes_)
```

Вероятностное представление может помочь в *задаче ранжирования*, где требуется не просто выдать метку, но и выстроить набор объектов в некотором порядке. С последним как раз может помочь вычисляемая моделью вероятность. Модели ранжирования часто используются в поисковых системах.

Также стоит упомянуть, что для упорядочения также подойдет и какая-то другая вычисляемая величина, от которой зависит выходная вероятность. Как, например, *отступ* в логистической регрессии, который к тому же вычислительно легче.

ROC-кривая

ROC-кривая (*receiver operating characteristic curve*) — график, позволяющий оценить качество бинарной классификации. Отображает соотношение между оценками *TPR* и *FPR* при варьировании порога решающего правила.

True Positive Rate (TPR) — отражает отношение количества объектов, которые модель *правильно классифицировала как Positive*, к общему числу объектов, которые *имеют класс Positive*. Отношение выглядит следующим образом

$$TPR = \frac{TP}{TP + FN}$$

Как можно заметить — это тот же *recall*, но в рамках ROC-кривой имеющий другое название. Также такую оценку называют **чувствительностью** модели классификации. Она показывает вероятность того, что объект класса *Positive* будет отмечен как *Positive*, или для реального примера — что больной субъект будет классифицирован именно как больной.

False Positive Rate (FPR) — доля объектов, ошибочно классифицированных как *Positive*, от общего количества объектов, не несущих целевой признак, т.е. принадлежащих классу *Negative*

$$FPR = \frac{FP}{TN + FP}$$

Обратная оценка

$$1 - FPR = \frac{TN}{TN + FP}$$

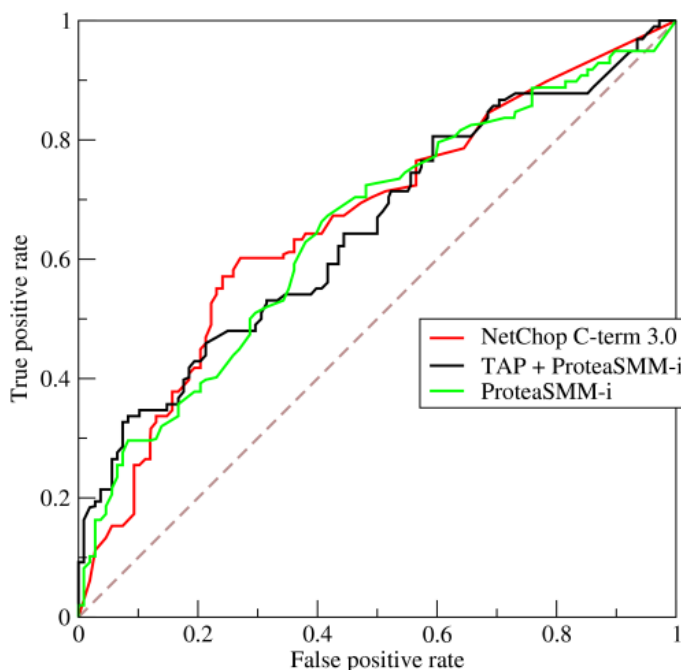
называется **специфичностью** модели и показывает вероятность отнесения объекта класса *Negative* к его истинному классу *Negative*, или же то, не больной субъект будет классифицирован именно как не больной. Можно сказать, что это есть *полнота* для класса *Negative*.

Под **порогом решающего правила** понимается порог для выходного значения модели, преодоление которого означает, что объект относится к целевому классу. Например, в случае логистической регрессии, стандартным порогом является 0.5, то есть если выходное значение логистической регрессии больше 0.5 — то объект помечается классом 1, если меньше — классом 0.

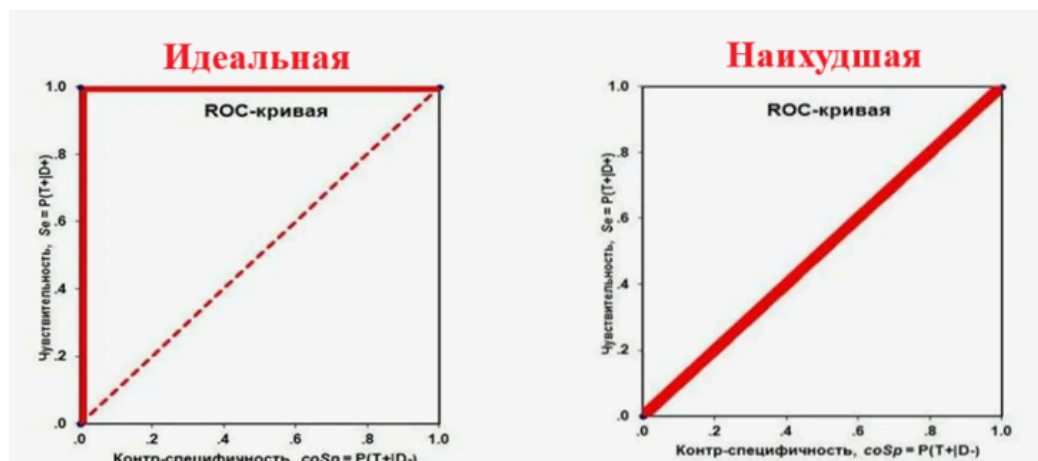
Таким образом, построение ROC-кривой происходит по следующему алгоритму:

1. Для всех объектов из *тестовой выборки* модель вычисляет значение, по которому определяется класс объекта
2. Объекты сортируются в порядке убывания этих значений
3. Задается начальный порог, как самое большое значение из всех полученных
4. Подсчитываются оценки TPR и FPR и ставится первая точка на плоскости
5. Дальнейшие точки графика вычисляются при изменении порога, который принимает полученные ранее значения в порядке убывания

В конце должна получиться кривая, идущая от точки $(0; 0)$ к точке $(1; 1)$



У идеальной модели найдется такой порог, при котором она будет со 100% вероятностью правильно классифицировать объекты, а ее ROC-кривая сначала поднимется к $TPR = 1$ и затем уйдет в точку $(1; 1)$. Наихудшая и непригодная к эксплуатации модель нарисует прямую от $(0; 0)$ до $(1; 1)$, то есть какой бы порог в модели не был взят, вероятность точного предсказания будет 50 на 50. Кривые реальных моделей располагаются между двумя этими графиками.



В качестве численной оценки, отражающей результаты построения *ROC-кривой*, используют площадь под ней — ***ROC-AUC (Area Under Curve)***.

Координаты точек для построения *ROC-кривой* рассчитывают следующим образом

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba[:,
1])
```

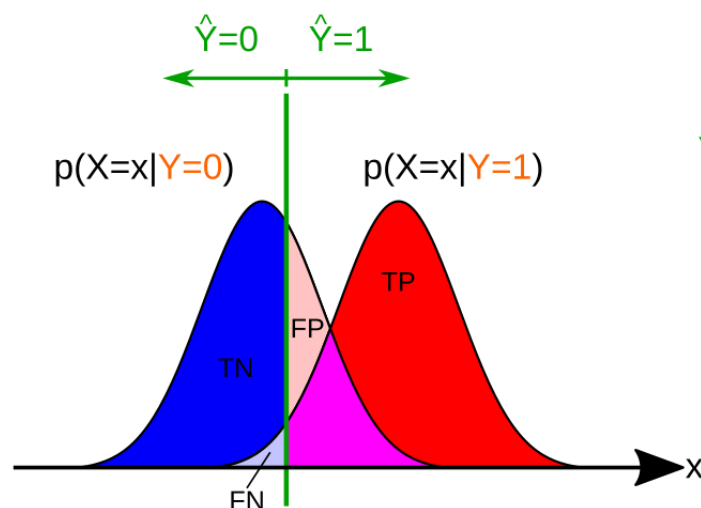
Метод возвращает три массива значений: *fpr* и *tpr* — координаты точек кривой, соответствующие оценкам *TPR* и *FPR* при изменяющемся *пороге решающего правила*, значения которого записаны в *thresholds*. Для отображения кривой используйте следующий код

```
plt.plot(fpr, tpr, marker='o')
plt.ylim([0,1.1])
plt.xlim([0,1.1])
plt.ylabel('TPR')
plt.xlabel('FPR')
plt.title('ROC curve')
```

Для получения метрики ***ROC-AUC*** используется метод из *sklearn.metrics*

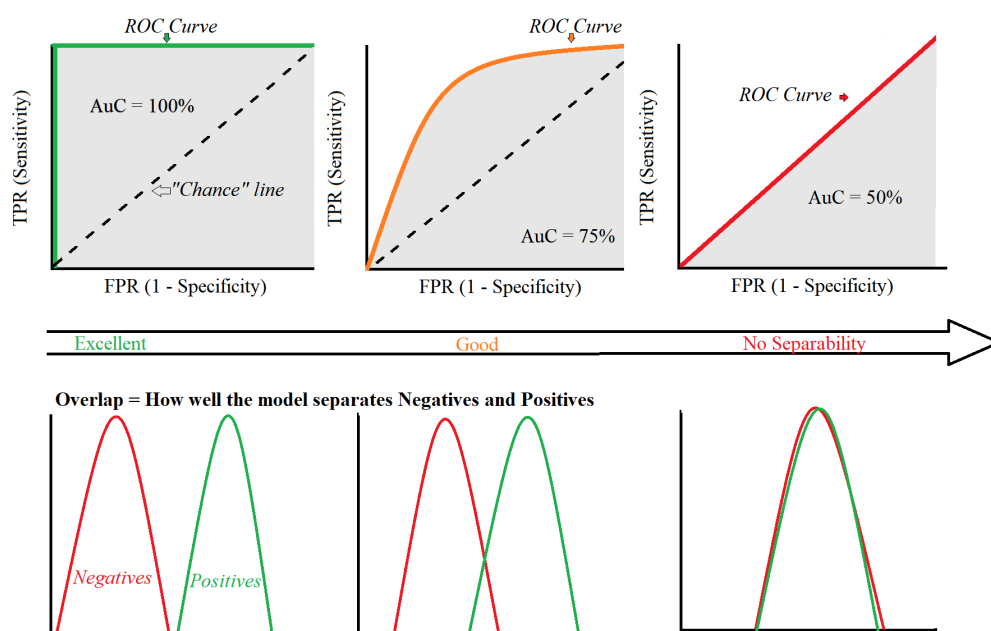
```
from sklearn.metrics import auc
auc_metric = auc(fpr, tpr)
```

В целом, *ROC-кривая* характеризует положение относительно друг друга плотностей распределения выходных значений модели для объектов классов *Positive* и *Negative* из тестовой выборки, ***то есть насколько хорошо модель разделяет объекты классов Positive и Negative***:



Ось X соответствует выходным значениям модели. Левая плотность демонстрирует плотность распределения выходных значений для объектов класса *Negative*, правая — *Positive*. Зеленая черта же отвечает за *порог решающего правила*. В процессе построения *ROC-кривой* эта черта движется справа налево.

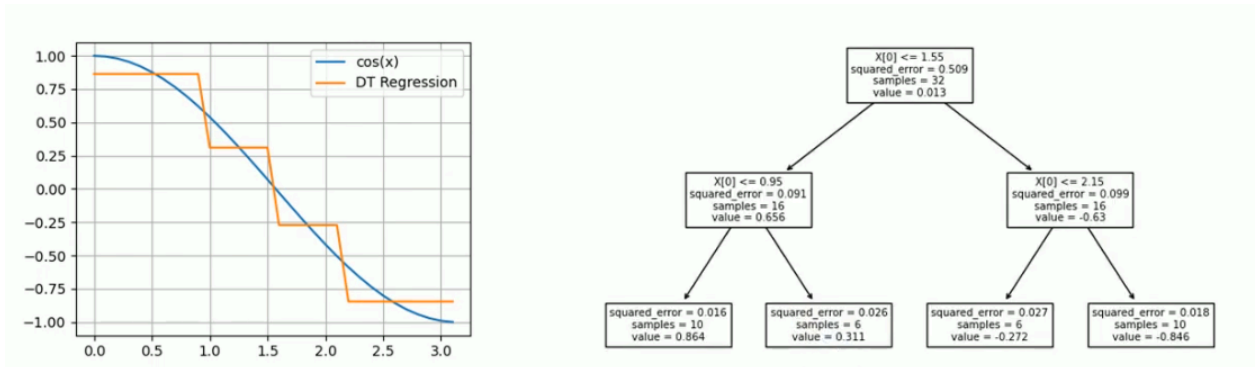
Ниже представлены плотности распределения и соответствующие им *ROC-кривые* для идеальной, хорошей и непригодной моделей



Решение задачи регрессии деревом решений

Дерево решений в задаче регрессии имеет одну особенность: ответы модели, хранящиеся в листах, являются константами. То есть, грубо говоря, выход модели не рассчитывается для входного наблюдения x , а для него выдается наиболее подходящее константное значение. Ниже представлен

пример обученного на значениях косинуса дерева решений с ограниченной глубиной 2



Из этого возникает два вопроса:

- Как разделять обучающую выборку при построении дерева?
- Какие значения хранить как ответы в листовых узлах?

Начнем со второго вопроса. Пусть в листовую вершину из обучающей выборки попало множество объектов и ответов для них $R \subset \{X^{train}, Y^{train}\}$. Тогда наилучший ответ в k -ом листе можно рассчитать минимизировав либо среднеабсолютную, либо среднеквадратичную ошибку

$$H_k(R) = \frac{1}{|R|} \sum_{(x^i, y^i) \in R} |a(x^i) - y^i| \rightarrow \min$$

$$H_k(R) = \frac{1}{|R|} \sum_{(x^i, y^i) \in R} (a(x^i) - y^i)^2 \rightarrow \min$$

Но так как значение в k -ой листовой вершине константное, то выход модели можно переобозначить как $a(x^i) = b_k$ и выражение примет вид

$$H_k(R) = \frac{1}{|R|} \sum_{(x^i, y^i) \in R} (b_k - y^i)^2 \rightarrow \min$$

Наилучшим же значением в листовой вершине в таком случае будет среднее всех ответов объектов обучающей выборки, которые попали в этот листовой узел

$$b_k = \frac{1}{|R|} \sum_{y^i \in R} y^i$$

Тогда такой критерий как

$$Avg(R) = \frac{1}{|R|} \sum_{y^i \in R} y^i$$

$$H(R) = \sum_{y^i \in R} (Avg(R) - y^i)^2$$

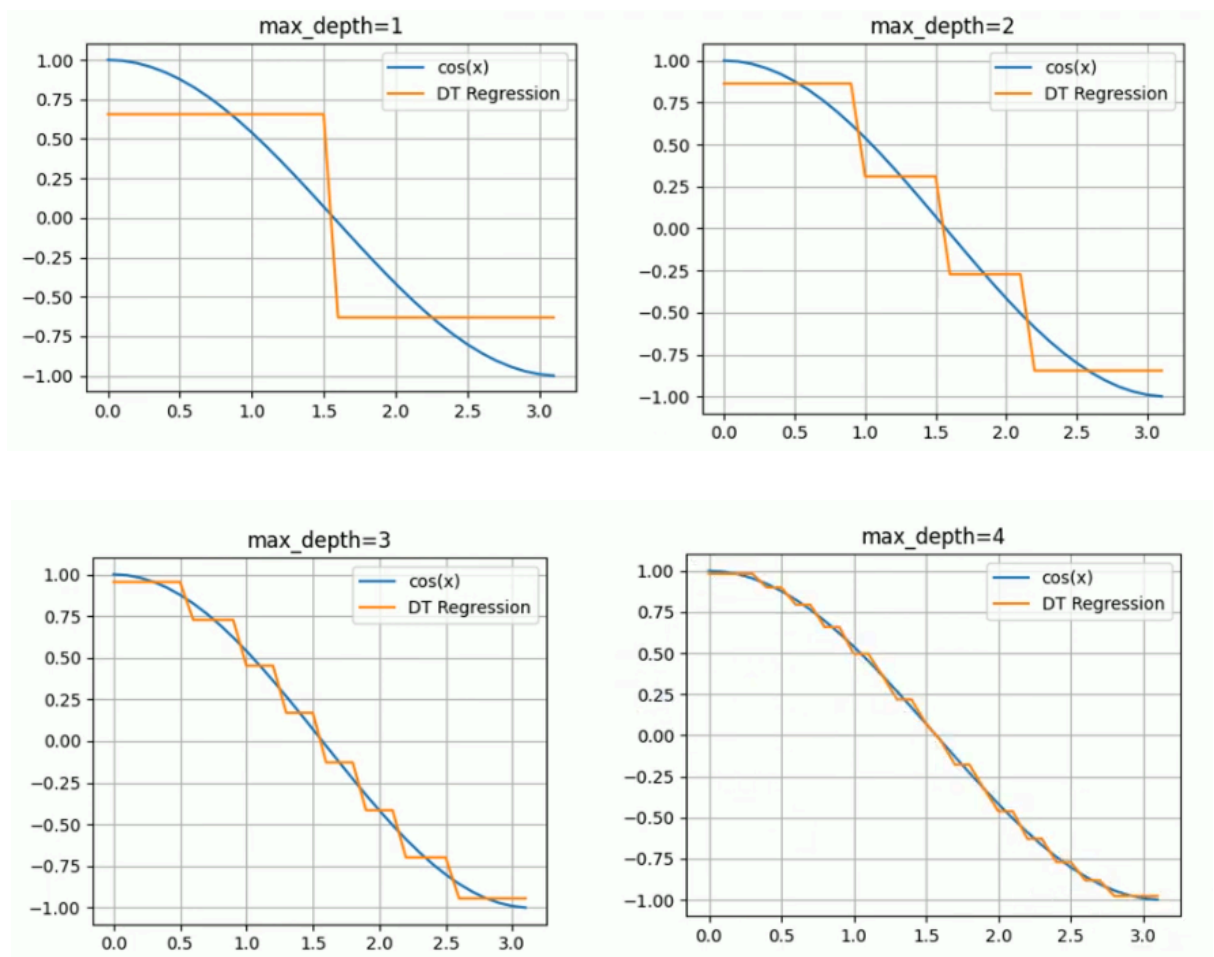
можно использовать для расчета информационного выигрыша при разделении выборки на k -ом узле в процессе обучения бинарного дерева решений

$$IG_k(j, a_k) = H(R_k) - \left(\frac{|R_l|}{|R_k|} H(R_l) + \frac{|R_r|}{|R_k|} H(R_r) \right) \rightarrow \max$$

где R_k — часть обучающей выборки, попавшей в узел k ; R_l и R_r — подмножества, полученные после разбиения R_k предикатом для j -го признака с порогом a_k . То есть, в данном случае, информационный выигрыш характеризует то, насколько снизилась средняя ошибка при добавлении дочерних узлов на основе предиката относительно ошибки в родительском узле.

Таким образом, при построении дерева решений для задачи регрессии, в вершине дерева подбирается такой предикат, который делил бы выборку из объектов обучающих данных, попавших в эту вершину, таким образом, чтобы взвешенная сумма *средних ошибок* в дочерних узлах была минимальна.

Из этого можно сделать еще один вывод — добавление дочерних узлов уменьшает среднюю ошибку *всей модели*. То есть увеличение максимальной глубины модели уменьшит её среднюю ошибку, но скорее всего это также приведет к переобучению при наличии шума в обучающих данных. Этот факт можно увидеть на следующем примере увеличения максимальной глубины дерева решений



Библиотека `scikit-learn` предлагает модель [***DecisionTreeRegressor***](#) для решения этой задачи.

```
from sklearn.tree import DecisionTreeRegressor
dt_regressor_model = DecisionTreeRegressor()
```

Модель также имеет гиперпараметры для ограничения структуры дерева и для настройки алгоритма решения регрессии. Для вывода графа дерева на экран, как было показано в примере выше, используйте

```
from sklearn.tree import DecisionTreeRegressor
tree.plot_tree(dt_model)
plt.show()
```

Итог

Преимущества решающих деревьев:

- Интерпретируемость и простота визуализации: можно отобразить граф дерева и явно понять, как оно принимает решение о выдаче метки класса

- Может работать как с категориальными, так и с числовыми признаками
- Допустимы пропуски в данных
- Не требует нормализации данных

Недостатки:

- При использовании жадных стратегий построения дерева высока вероятность переобучить модель
- Листья, находящиеся на большой глубине дерева, будут иметь низкую статистическую надежность из-за малой выборки в них
- Высокая чувствительность к шуму в объектах выборки
- Могут быть неустойчивыми: малые изменения в данных могут привести к сильным изменениям в модели

Задание

1. Использовать датасет, подготовленный на первой лабораторной работе
2. Решить задачу регрессии для одного из непрерывных признаков в датасете. Оценить работу регрессионной модели
3. Решить задачу классификации и оценить работу модели с помощью ROC-кривой
4. Выгрузить результат работы на Github